

小结

305

类是 C++ 语言中最基本的特性。类允许我们为自己的应用定义新类型，从而使得程序更加简洁且易于修改。

类有两项基本能力：一是数据抽象，即定义数据成员和函数成员的能力；二是封装，即保护类的成员不被随意访问的能力。通过将类的实现细节设为 `private`，我们就能完成类的封装。类可以将其他类或者函数设为友元，这样它们就能访问类的非公有成员了。

类可以定义一种特殊的成员函数：构造函数，其作用是控制初始化对象的方式。构造函数可以重载，构造函数应该使用构造函数初始值列表来初始化所有数据成员。

类还能定义可变或者静态成员。一个可变成员永远都不会是 `const`，即使在 `const` 成员函数内也能修改它的值；一个静态成员可以是函数也可以是数据，静态成员存在于所有对象之外。

术语表

抽象数据类型 (abstract data type) 封装 (隐藏) 了实现细节的数据结构。

访问说明符 (access specifier) 包括关键字 `public` 和 `private`。用于定义成员对类的用户可见还是只对类的友元和成员可见。在类中说明符可以出现多次，每个说明符的有效范围从它自身开始，到下一个说明符为止。

聚合类 (aggregate class) 只含有公有成员的类，并且没有类内初始值或者构造函数。聚合类的成员可以用花括号括起来的初始值列表进行初始化。

类 (class) C++ 提供的自定义数据类型的机制。类可以包含数据、函数和类型成员。一个类定义一种新的类型和一个新的作用域。

类的声明 (class declaration) 首先是关键字 `class` (或者 `struct`)，随后是类名以及分号。如果类已经声明而尚未定义，则它是一个不完全类型。

class 关键字 (class keyword) 用于定义类的关键字，默认情况下成员是 `private` 的。

类的作用域 (class scope) 每个类定义一个作用域。类作用域比其他作用域更加复

杂，类中定义的成员函数甚至有可能使用定义语句之后的名字。

常量成员函数 (const member function) 一个成员函数，在其中不能修改对象的普通 (即既不是 `static` 也不是 `mutable`) 数据成员。`const` 成员的 `this` 指针是指向常量的指针，通过区分函数是否是 `const` 可以进行重载。

构造函数 (constructor) 用于初始化对象的一种特殊的成员函数。构造函数应该给每个数据成员都赋一个合适的初始值。

构造函数初始值列表 (constructor initializer list) 说明一个类的数据成员的初始值，在构造函数体执行之前首先用初始值列表中的值初始化数据成员。未经初始值列表初始化的成员将被默认初始化。

转换构造函数 (converting constructor) 可以用一个实参调用的非显式构造函数。这样的函数隐式地将参数类型转换成类类型。

数据抽象 (data abstraction) 着重关注类型接口的一种编程技术。数据抽象令程序员可以忽略类型的实现细节，只关注类型执行的操作即可。数据抽象是面向对象编程和泛型编程的基础。

306

默认构造函数 (default constructor) 当没有提供任何实参时使用的构造函数。

委托构造函数 (delegating constructor) 委托构造函数的初始值列表只有一个入口, 指定类的另一个构造函数执行初始化操作。

封装 (encapsulation) 分离类的实现与接口, 从而隐藏了类的实现细节。在 C++ 语言中, 通过把实现部分设为 `private` 完成封装的任务。

显式构造函数 (explicit constructor) 可以用一个单独的实参调用但是不能用于隐式转换的构造函数。通过在构造函数的声明之前加上 `explicit` 关键字就可以将其声明成显式构造函数。

前向声明 (forward declaration) 对尚未定义的名字的声明, 通常用于表示位于类定义之前的类声明。参见“不完全类型”。

友元 (friend) 类向外部提供其非公有成员访问权限的一种机制。友元的访问权限与成员函数一样。友元可以是类, 也可以是函数。

实现 (implementation) 类的成员 (通常是私有的), 定义了不希望为使用类类型的代码所用的数据及任何操作。

不完全类型 (incomplete type) 已经声明但是尚未定义的类型。不完全类型不能用于定义变量或者类的成员, 但是用不完全类型定义指针或者引用是合法的。

接口 (interface) 类型提供的 (公有) 操作。通常情况下, 接口不包含数据成员。

成员函数 (member function) 类的函数成员。普通的成员函数通过隐式的 `this` 指针与类的对象绑定在一起; 静态成员函数不与对象绑定在一起也没有 `this` 指针。

成员函数可以重载, 此时隐式的 `this` 指针参与函数匹配的过程。

可变数据成员 (mutable data member) 这种成员永远不是 `const`, 即使它属于 `const` 对象。在 `const` 函数内可以修改可变数据成员。

名字查找 (name lookup) 根据名字的使用寻找匹配的声明的过程。

私有成员 (private member) 定义在 `private` 访问说明符之后的成员, 只能被类的友元或者类的其他成员访问。数据成员以及仅供类本身使用而不作为接口的功能函数一般设为 `private`。

公有成员 (public member) 定义在 `public` 访问说明符之后的成员, 可以被类的所有用户访问。通常情况下, 只有实现类的接口的函数才被设为 `public`。

struct 关键字 (struct keyword) 用于定义类的关键字, 默认情况下成员是 `public` 的。

合成默认构造函数 (synthesized default constructor) 对于没有显式地定义任何构造函数的类, 编译器为其创建 (合成) 的默认构造函数。该构造函数检查类的数据成员, 如果提供了类内初始值, 就用它执行初始化操作; 否则就对数据成员执行默认初始化。

this 指针 (this pointer) 是一个隐式的值, 作为额外的实参传递给类的每个非静态成员函数。`this` 指针指向代表函数调用者的对象。

= default 一种语法形式, 位于类内部默认构造函数声明语句的参数列表之后, 要求编译器生成构造函数, 而不管类是否已经有了其他构造函数。

第 II 部分

C++ 标准库

内容

第 8 章 IO 库.....	277
第 9 章 顺序容器.....	291
第 10 章 泛型算法.....	335
第 11 章 关联容器.....	373
第 12 章 动态内存.....	399

随着 C++ 版本的一次次修订，标准库也在不断成长。确实，新的 C++ 标准中有三分之二的文本都用来描述标准库。虽然我们不能深入讨论所有标准库设施，但有些核心库设施是每个 C++ 程序员都应该熟练掌握的，第二部分将介绍这些内容。

我们首先在第 8 章中介绍基本的 IO 库设施。除了使用标准库读写与控制台窗口相关联的流之外，我们还将学习其他一些库类型，可以帮助我们读写命名文件以及完成到 `string` 对象的内存 IO 操作。

标准库的核心是很多容器类和一族泛型算法，这些设施能帮助我们编写简洁高效的程序。标准库会去关注那些簿记操作的细节，特别是内存管理，这样我们的程序就可以将全部注意力投入到需要求解的问题上。

我们在第 3 章中已经介绍了容器类型 `vector`，在第 9 章中将介绍更多 `vector` 相关的内容，这一章也会涉及其他顺序容器。我们还会介绍更多 `string` 类型所支持的操作，可以将 `string` 看作一种只包含字符元素的特殊容器。`string` 支持很多容器操作，但不是全部。

第 10 章介绍泛型算法。这类算法通常在顺序容器一定范围内的元素上或其他类型的序列上进行操作。算法库为各种经典算法提供了高效的实现，如排序和搜索算法，还提供了其他一些常用操作。例如，标准库提供了 `copy` 算法，完成一个序列到另一个序列的元素拷贝；还提供了 `find` 算法，实现给定元素的查找，等等。泛型算法的通用性体现在两个层面：可应用于不同类型的序列；对序列中元素的类型限制小，大多数类型都是允许的。

标准库还提供了一些关联容器，第 11 章介绍这部分内容。关联容器中的元素是通过关键字来访问的。关联容器支持很多顺序容器的操作，也定义了一些自己特有的操作。

第 12 章是第二部分的一章，这一章介绍动态内存管理相关的一些语言特性和库设施。这一章介绍智能指针的一个标准版本，它是新标准库中最重要的类之一。通过使用智能指针，我们可以大幅度提高使用动态内存的代码的鲁棒性。这一章最后将给出一个较大的例子，使用了第II部分介绍的所有标准库设施。

第 8 章

IO 库

内容

8.1 IO 类.....	278
8.2 文件输入输出	283
8.3 string 流.....	287
小结	290
术语表.....	290

C++语言不直接处理输入输出，而是通过一族定义在标准库中的类型来处理 IO。这些类型支持从设备读取数据、向设备写入数据的 IO 操作，设备可以是文件、控制台窗口等。还有一些类型允许内存 IO，即，从 string 读取数据，向 string 写入数据。

IO 库定义了读写内置类型值的操作。此外，一些类，如 string，通常也会定义类似的 IO 操作，来读写自己的对象。

本章介绍 IO 库的基本内容。后续章节会介绍更多 IO 库的功能：第 14 章将会介绍如何编写自己的输入输出运算符，第 17 章将会介绍如何控制输出格式以及如何对文件进行随机访问。